`

Compiler Design

Case Study on Optimization Techniques

CB.SC.U4CSE23705 – Aswin potta

CB.SC.U4CSE23728 – K Manikanta sai vignan

CB.SC.U4CSE23733 – Nandakishor

CB.SC.U4CSE23736 – P.Tarun

`

1.Inter-Procedural Constant Propagation (IPA-CP) and Folding

1. The original program

```
static int __attribute__((noinline)) multiply(int a, int b) {
return a * b;
}

int main() {
return multiply(5, 5) + multiply(5, 10);
```

2. The assembly code of the original program. (with – O0 option).

```
multiply:
push rbp
mov rbp, rsp
mov DWORD PTR [rbp-4], edi
mov DWORD PTR [rbp-8], esi
mov eax, DWORD PTR [rbp-4]
imul eax, DWORD PTR [rbp-8]
pop rbp
ret
main:
push rbp
mov rbp, rsp
push rbx
mov esi, 5
mov edi, 5
call multiply
mov ebx, eax
mov esi, 10
mov edi, 5
call multiply
add eax, ebx
mov rbx, QWORD PTR [rbp-8]
leave
ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cf_O0.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 161
Ticks             : 1613669
TotalDays         : 1.8676724537037E-06
TotalHours        : 4.48241388888889E-05
TotalMinutes      : 0.00268944833333333
TotalSeconds      : 0.1613669
TotalMilliseconds : 161.3669
                                                    Ln 1, Col 1    Spaces: 4
```

3. The assembly code of the optimized program (with – O1 option).

```
multiply:
mov eax, edi
imul eax, esi
ret
main:
push rbx
mov esi, 5
mov edi, 5
call multiply
mov ebx, eax
mov esi, 10
mov edi, 5
call multiply
add eax, ebx
pop rbx
ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cf_O1.exe }
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 38
Ticks             : 388089
TotalDays         : 4.49177083333333E-07
TotalHours        : 1.078025E-05
TotalMinutes      : 0.000646815
TotalSeconds      : 0.0388089
TotalMilliseconds : 38.8089
```

4. The assembly code of the optimized program (with – O2 option).

```
multiply.constprop.0:
lea eax, [rdi+rdi*4]
ret
main:
mov edi, 5
call multiply.constprop.0
mov edi, 10
mov edx, eax
call multiply.constprop.0
add eax, edx
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cf_O2.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 42
Ticks             : 425460
TotalDays         : 4.92430555555555E-07
TotalHours        : 1.18183333333333E-05
TotalMinutes      : 0.0007091
TotalSeconds      : 0.042546
TotalMilliseconds : 42.546
```

`

5. The assembly code of the optimized program (with – O3 option).

```
multiply.constprop.0:
lea eax, [rdi+rdi*4]
ret
main:
mov edi, 5
call multiply.constprop.0
mov edi, 10
mov edx, eax
call multiply.constprop.0
add eax, edx
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cf_O3.exe }


 Days              : 0
 Hours             : 0
 Minutes           : 0
 Seconds           : 0
 Milliseconds      : 271
 Ticks             : 2719753
 TotalDays         : 3.14786226851852E-06
 TotalHours        : 7.55486944444444E-05
 TotalMinutes      : 0.00453292166666667
 TotalSeconds      : 0.2719753
 TotalMilliseconds : 271.9753
```

6. The assembly code of the optimized program (with – Os option).

```
multiply.constprop.0:
lea eax, [rdi+rdi*4]
ret
main:
mov edi, 5
call multiply.constprop.0
mov edi, 10
mov edx, eax
call multiply.constprop.0
add eax, edx
ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cf_Os.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 55
Ticks             : 556188
TotalDays         : 6.43736111111111E-07
TotalHours        : 1.54496666666667E-05
TotalMinutes      : 0.00092698
TotalSeconds      : 0.0556188
                                                                Ln 1, Col 1    Spaces
```

7. Inference: Analysis of Inter-Procedural Constant Propagation

At -O0, the compiler generates a literal translation involving costly imul instructions and overhead-heavy activation records. However, at -O2 and above, the optimizer performs a call-graph analysis and identifies that the formal parameter a is a constant value of 5 across all call sites. This triggers the creation of a specialized function version, multiply.constprop.0, where the multiplication is replaced by a high-speed Strength Reduction transformation. By using the lea instruction to calculate [rdi+rdi*4] (representing b+4b=5b), the compiler eliminates ALU-intensive multiplication in favor of a single-cycle shift-and-add operation. This significantly reduces the cycles per instruction (CPI) and total instruction count, highlighting how inter-procedural analysis optimizes cross-function data flow.

`

2.Global Dead-Code Elimination (DCE) and Unreachable-Code Removal

1. The original program

```
int square(int num) {
return num * num;
}int test(int x) {
int y = x * 100;
int z = x + 50;
y = 20;
return y;
}
```

2. The assembly code of the original program. (with – O0 option).

```
square:
push rbp
mov rbp, rsp
mov DWORD PTR [rbp-4], edi
mov eax, DWORD PTR [rbp-4]
imul eax, eax
pop rbp
ret
test:
push rbp
mov rbp, rsp
mov DWORD PTR [rbp-20], edi
mov eax, DWORD PTR [rbp-20]
imul eax, eax, 100
mov DWORD PTR [rbp-4], eax
mov eax, DWORD PTR [rbp-20]
add eax, 50
mov DWORD PTR [rbp-8], eax
mov DWORD PTR [rbp-4], 20
mov eax, DWORD PTR [rbp-4]
pop rbp
ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\dce_O0.exe

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 460
Ticks             : 4605725
TotalDays         : 5.33070023148148E-06
TotalHours        : 0.000127936805555556
TotalMinutes      : 0.00767620833333333
TotalSeconds      : 0.4605725
TotalMilliseconds : 460.5725
```

3. The assembly code of the optimized program (with – O1 option).

```
square:
imul edi, edi
mov eax, edi
ret
test:
mov eax, 20
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\dce_O1.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 394
Ticks             : 3947872
TotalDays         : 4.5692962962963E-06
TotalHours        : 0.000109663111111111
TotalMinutes      : 0.00657978666666667
TotalSeconds      : 0.3947872
TotalMilliseconds : 394.7872
```

4. The assembly code of the optimized program (with – O2 option).

```
square:
imul edi, edi
mov eax, edi
ret
test:
```

`

```
mov eax, 20
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\dce_O2.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 295
Ticks             : 2956899
TotalDays         : 3.42233680555556E-06
TotalHours        : 8.21360833333333E-05
TotalMinutes      : 0.004928165
TotalSeconds      : 0.2956899
TotalMilliseconds : 295.6899
```

5. The assembly code of the optimized program (with – O3 option).

```
square:
imul edi, edi
mov eax, edi
ret
test:
mov eax, 20
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\dce_O3.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 378
Ticks             : 3789352
TotalDays         : 4.38582407407407E-06
TotalHours        : 0.000105259777777778
TotalMinutes      : 0.00631558666666667
TotalSeconds      : 0.3789352
TotalMilliseconds : 378.9352
```

`

6. The assembly code of the optimized program (with – Os option).

```
square:
imul edi, edi
mov eax, edi
ret
test:
mov eax, 20
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\dce_Os.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 396
Ticks             : 3966997
TotalDays         : 4.59143171296296E-06
TotalHours        : 0.000110194361111111
TotalMinutes      : 0.00661166166666667
TotalSeconds      : 0.3966997
TotalMilliseconds : 396.6997
                                                    Ln 1, Col 1    Spaces: 4    UTF-
```

7. Inference: Analysis of Global Dead-Code Elimination (DCE) and Unreachable-Code Removal

At -O0, the function test retains all intermediate computations: y = x * 100 and z = x + 50. However, both values are overwritten or never used in any observable context. Specifically, y is reassigned to 20 before being returned, and z has no subsequent use. These computations are therefore dead under liveness analysis.

At -O1 and higher, the compiler performs global data-flow analysis and determines that the only live value at function exit is the constant 20. Consequently:

The initial computation of y (x * 100) is eliminated because its value is overwritten before any use.

The computation of z is removed entirely since it has no uses (pure dead code).

All associated memory stores and loads are pruned.

`

Additionally, control-flow simplification collapses the function body into a single basic block with a constant return value. This effectively removes any implicit unreachable execution paths tied to the eliminated computations.

The resulting assembly reduces test to mov eax, 20; ret, minimizing instruction count, register pressure, and memory traffic. This demonstrates how DCE, guided by liveness and def-use chains, aggressively eliminates non-contributing computations, enabling maximal simplification of the program while preserving semantics.

`

3.Global Common Subexpression Elimination (GVN/CSE)

1. The Original Program

```
__attribute__((noinline))
int test(int a, int b, int c) {
    volatile int x = (a * b) + c;
    volatile int y = (a * b) - c;
    volatile int z = (a * b) * c;
    return x + y + z;
}
```

2. The assembly code of the original program. (with – O0 option).

```
test:
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-20], edi
        mov     DWORD PTR [rbp-24], esi
        mov     DWORD PTR [rbp-28], edx
        mov     eax, DWORD PTR [rbp-20]
        imul    eax, DWORD PTR [rbp-24]
        mov     edx, eax
        mov     eax, DWORD PTR [rbp-28]
        add     eax, edx
        mov     DWORD PTR [rbp-4], eax
        mov     eax, DWORD PTR [rbp-20]
        imul    eax, DWORD PTR [rbp-24]
        sub     eax, DWORD PTR [rbp-28]
        mov     DWORD PTR [rbp-8], eax
        mov     eax, DWORD PTR [rbp-20]
        imul    eax, DWORD PTR [rbp-24]
        imul    eax, DWORD PTR [rbp-28]
        mov     DWORD PTR [rbp-12], eax
        mov     edx, DWORD PTR [rbp-4]
        mov     eax, DWORD PTR [rbp-8]
        add     edx, eax
        mov     eax, DWORD PTR [rbp-12]
        add     eax, edx
        pop     rbp
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cse_O0.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 353
Ticks             : 3536028
TotalDays         : 4.092625E-06
TotalHours        : 9.8223E-05
TotalMinutes      : 0.00589338
TotalSeconds      : 0.3536028
TotalMilliseconds : 353.6028
```

3. The assembly code of the optimized program (with – O1 option).

```
test:
        imul    edi, esi
        lea     eax, [rdi+rdx]
        mov     DWORD PTR [rsp-4], eax
        mov     eax, edi
        sub     eax, edx
        mov     DWORD PTR [rsp-8], eax
        imul    edi, edx
        mov     DWORD PTR [rsp-12], edi
        mov     eax, DWORD PTR [rsp-4]
        mov     ecx, DWORD PTR [rsp-8]
        mov     edx, DWORD PTR [rsp-12]
        add     eax, ecx
        add     eax, edx
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cse_O1.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 614
Ticks             : 6142205
TotalDays         : 7.10903356481481E-06
TotalHours        : 0.000170616805555556
TotalMinutes      : 0.0102370083333333
TotalSeconds      : 0.6142205
TotalMilliseconds : 614.2205
```

`

4. The assembly code of the optimized program (with – O2 option).

```
test:
        imul    edi, esi
        lea     eax, [rdi+rdx]
        mov     DWORD PTR [rsp-12], eax
        mov     eax, edi
        imul    edi, edx
        sub     eax, edx
        mov     DWORD PTR [rsp-8], eax
        mov     DWORD PTR [rsp-4], edi
        mov     eax, DWORD PTR [rsp-12]
        mov     ecx, DWORD PTR [rsp-8]
        mov     edx, DWORD PTR [rsp-4]
        add     eax, ecx
        add     eax, edx
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cse_O2.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 571
Ticks             : 5711282
TotalDays         : 6.61028009259259E-06
TotalHours        : 0.000158646722222222
TotalMinutes      : 0.00951880333333333
TotalSeconds      : 0.5711282
TotalMilliseconds : 571.1282
```

5. The assembly code of the optimized program (with – O3 option).

```
test:
        imul    edi, esi
        lea     eax, [rdi+rdx]
        mov     DWORD PTR [rsp-12], eax
        mov     eax, edi
        imul    edi, edx
        sub     eax, edx
```

`

```
        mov     DWORD PTR [rsp-8], eax
        mov     DWORD PTR [rsp-4], edi
        mov     eax, DWORD PTR [rsp-12]
        mov     ecx, DWORD PTR [rsp-8]
        mov     edx, DWORD PTR [rsp-4]
        add     eax, ecx
        add     eax, edx
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cse_O3.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 713
Ticks             : 7138910
TotalDays         : 8.26262731481481E-06
TotalHours        : 0.000198303055555556
TotalMinutes      : 0.0118981833333333
TotalSeconds      : 0.713891
TotalMilliseconds : 713.891
                                                    Ln 1, Col 1    Spaces: 4
```

6. The assembly code of the optimized program (with – Os option).

```
test:
        imul    edi, esi
        lea     eax, [rdi+rdx]
        mov     DWORD PTR [rsp-12], eax
        mov     eax, edi
        imul    edi, edx
        sub     eax, edx
        mov     DWORD PTR [rsp-8], eax
        mov     DWORD PTR [rsp-4], edi
        mov     eax, DWORD PTR [rsp-12]
        mov     ecx, DWORD PTR [rsp-8]
        mov     edx, DWORD PTR [rsp-4]
        add     eax, ecx
        add     eax, edx
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cse_Os.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 539
Ticks             : 5396399
TotalDays         : 6.24583217592593E-06
TotalHours        : 0.000149899972222222
TotalMinutes      : 0.00899399833333333
TotalSeconds      : 0.5396399
TotalMilliseconds : 539.6399
```

7. Inference: Analysis of Global Common Subexpression Elimination (GVN/CSE)

At **-O0**, the compiler directly translates the source, recomputing the common expression (a * b) multiple times using separate imul instructions, leading to redundant ALU operations and higher instruction count. At **-O1 and above**, Global Value Numbering (GVN) identifies the repeated expression and computes it once, reusing the result for subsequent operations. This removes redundant arithmetic, reduces execution pressure, and lowers dynamic instruction count. Since the program lacks loops or complex control flow, higher levels (-O2, -O3, -Os) do not apply additional transformations, resulting in identical optimized assembly beyond -O1.

`

4.SSA-Based Copy Propagation and Redundant ϕ-Elimination

1. The Original Program

```c
#include <stdio.h>

int main() {
    int x = 10;
    int y = x;       // copy: y is just x
    int z = y;       // copy: z is just y (i.e., x)
    int result;

    if (x > 5) {
        result = z;  // redundant phi candidate
    } else {
        result = z;  // same value on both paths
    }

    printf("%d\n", result);
    return 0;
}
```

2. The assembly code of the original program. (with – O0 option).

```asm
.LC0:
        .string "%d\n"
main:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        mov     DWORD PTR [rbp-8], 10
        mov     eax, DWORD PTR [rbp-8]
        mov     DWORD PTR [rbp-12], eax
        mov     eax, DWORD PTR [rbp-12]
        mov     DWORD PTR [rbp-16], eax
        cmp     DWORD PTR [rbp-8], 5
        jle     .L2
        mov     eax, DWORD PTR [rbp-16]
        mov     DWORD PTR [rbp-4], eax
        jmp     .L3
```

`

```
.L2:
        mov     eax, DWORD PTR [rbp-16]
        mov     DWORD PTR [rbp-4], eax
.L3:
        mov     eax, DWORD PTR [rbp-4]
        mov     esi, eax
        mov     edi, OFFSET FLAT:.LC0
        mov     eax, 0
        call    printf
        mov     eax, 0
        leave
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cp_O0.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 565
Ticks             : 5653652
TotalDays         : 6.5435787037037E-06
TotalHours        : 0.000157045888888889
TotalMinutes      : 0.00942275333333333
TotalSeconds      : 0.5653652
TotalMilliseconds : 565.3652
```

3. The assembly code of the original program. (with – O1 option).

```
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 10
        mov     edi, OFFSET FLAT:.LC0
        mov     eax, 0
        call    printf
        mov     eax, 0
        add     rsp, 8
        ret
```

Execution Time:

`

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cp_O1.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 625
Ticks             : 6259843
TotalDays         : 7.24518865740741E-06
TotalHours        : 0.000173884527777778
TotalMinutes      : 0.0104330716666667
TotalSeconds      : 0.6259843
```

4. The assembly code of the original program. (with – O2 option).

```
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 10
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        add     rsp, 8
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cp_O2.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 474
Ticks             : 4749595
TotalDays         : 5.49721643518519E-06
TotalHours        : 0.000131933194444444
TotalMinutes      : 0.00791599166666667
TotalSeconds      : 0.4749595
TotalMilliseconds : 474.9595
```

5. The assembly code of the original program. (with – O3 option).

`

```
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 10
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        add     rsp, 8
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cp_O3.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 484
Ticks             : 4841115
TotalDays         : 5.60314236111111E-06
TotalHours        : 0.000134475416666667
TotalMinutes      : 0.008068525
TotalSeconds      : 0.4841115
TotalMilliseconds : 484.1115
```

6. The assembly code of the original program. (with – Os option).

```
.LC0:
        .string "%d\n"
main:
        push    rax
        mov     esi, 10
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        pop     rdx
        ret
```

Execution Time:

`

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\cp_Os.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 638
Ticks             : 6384692
TotalDays         : 7.38968981481481E-06
TotalHours        : 0.000177352555555556
TotalMinutes      : 0.0106411533333333
TotalSeconds      : 0.6384692
TotalMilliseconds : 638.4692
```

7. **Inference: Analysis of SSA-Based Copy Propagation and Redundant ϕ-Elimination**

   At **-O0**, the compiler produces a literal translation where every copy assignment (y = x, z = y) results in redundant load-store pairs across separate stack slots, and both branches of the if-else generate identical stores to result — a classic materialization of a redundant ϕ-node at the CFG merge point. At **-O1**, SSA-based copy propagation identifies y and z as pure aliases of x, collapses the entire copy chain to the constant 10, eliminates the redundant ϕ-node since both branch paths carry the same value, and removes the if-else entirely — reducing the function to a direct printf call with esi = 10. At **-O2**, the only additional change is replacing mov eax, 0 with xor eax, eax, a peephole idiom that is shorter to encode and breaks false register dependencies for better pipeline efficiency. **-O3** produces no further changes, as the optimization ceiling for this scalar function is already reached at -O1. At **-Os**, the sub rsp, 8 / add rsp, 8 alignment pair is swapped for push rax / pop rdx, achieving the same ABI-required 16-byte stack alignment in fewer instruction bytes — the only size-specific trade-off visible in this program.

`

5.Loop-Invariant Code Motion (LICM) and Partial Redundancy Elimination (PRE)

1. The Original Code

```c
#include <stdio.h>

int main() {
    int a = 4, b = 3;
    int result = 0;

    for (int i = 0; i < 10; i++) {
        int t = a * b;       // loop-invariant: a and b never change inside loop
        result += t + i;     // t is hoisted out by LICM; i is loop-variant
    }

    printf("%d\n", result);
    return 0;
}
```

2. The assembly code of the original program. (with – O0 option).

```asm
.LC0:
        .string "%d\n"
main:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 32
        mov     DWORD PTR [rbp-12], 4
        mov     DWORD PTR [rbp-16], 3
        mov     DWORD PTR [rbp-4], 0
        mov     DWORD PTR [rbp-8], 0
        jmp     .L2
.L3:
        mov     eax, DWORD PTR [rbp-12]
        imul    eax, DWORD PTR [rbp-16]
        mov     DWORD PTR [rbp-20], eax
        mov     edx, DWORD PTR [rbp-20]
        mov     eax, DWORD PTR [rbp-8]
        add     eax, edx
        add     DWORD PTR [rbp-4], eax
```

`

```
        add     DWORD PTR [rbp-8], 1
.L2:
        cmp     DWORD PTR [rbp-8], 9
        jle     .L3
        mov     eax, DWORD PTR [rbp-4]
        mov     esi, eax
        mov     edi, OFFSET FLAT:.LC0
        mov     eax, 0
        call    printf
        mov     eax, 0
        leave
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\licm_O0.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 548
Ticks             : 5489488
TotalDays         : 6.35357407407407E-06
TotalHours        : 0.000152485777777778
TotalMinutes      : 0.00914914666666667
TotalSeconds      : 0.5489488
TotalMilliseconds : 548.9488
```

3. The assembly code of the original program. (with – O1 option).

```
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 165
        mov     edi, OFFSET FLAT:.LC0
        mov     eax, 0
        call    printf
        mov     eax, 0
        add     rsp, 8
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\licm_O1.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 480
Ticks             : 4802064
TotalDays         : 5.55794444444444E-06
TotalHours        : 0.000133390666666667
TotalMinutes      : 0.00800344
TotalSeconds      : 0.4802064
TotalMilliseconds : 480.2064
```

4. The assembly code of the original program. (with – O2 option).

```
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 165
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        add     rsp, 8
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\licm_O2.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 338
Ticks             : 3383197
TotalDays         : 3.91573726851852E-06
TotalHours        : 9.39776944444444E-05
TotalMinutes      : 0.00563866166666667
TotalSeconds      : 0.3383197
TotalMilliseconds : 338.3197
```

`

5. The assembly code of the original program. (with – O3 option).

```
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 165
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        add     rsp, 8
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\licm_O3.exe }


  Days              : 0
  Hours             : 0
  Minutes           : 0
  Seconds           : 0
  Milliseconds      : 595
  Ticks             : 5951327
  TotalDays         : 6.8881099537037E-06
  TotalHours        : 0.000165314638888889
  TotalMinutes      : 0.00991887833333333
  TotalSeconds      : 0.5951327
  TotalMilliseconds : 595.1327
```

6. The assembly code of the original program. (with – Os option).

```
.LC0:
        .string "%d\n"
main:
        push    rax
        mov     esi, 165
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        pop     rdx
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\licm_Os.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 560
Ticks             : 5607348
TotalDays         : 6.48998611111111E-06
TotalHours        : 0.000155759666666667
TotalMinutes      : 0.00934558
TotalSeconds      : 0.5607348
TotalMilliseconds : 560.7348
```

7. **Inference: Analysis of Loop-Invariant Code Motion (LICM) and Partial Redundancy Elimination (PRE)**

   At **-O0**, the compiler faithfully executes a * b on every iteration of the loop using a costly imul instruction inside .L3, recomputing the same value 10 times despite a and b never changing — a clear missed optimization that LICM is designed to eliminate. At **-O1**, the compiler applies LICM to identify that a * b = 12 is loop-invariant and hoists it out of the loop entirely, then PRE recognizes that result += 12 + i is partially redundant across all iterations and folds the entire loop into the compile-time constant 165 (sum of 12+0, 12+1, ... 12+9), reducing the loop from 10 dynamic iterations to zero — the loop ceases to exist in the binary. At **-O2**, the loop elimination result is identical, with the only change being mov eax, 0 replaced by xor eax, eax for the same pipeline and encoding efficiency reasons as before. **-O3** produces no further changes, as there is no loop left to vectorize or unroll — the optimization ceiling is already reached at -O1. At **-Os**, the sub/add rsp stack alignment pair is again replaced with push rax / pop rdx to save instruction bytes, consistent with the size-first strategy, while the core LICM and PRE results remain unchanged. The key takeaway is that LICM and PRE together form a powerful redundancy elimination pipeline — LICM reduces O(n) recomputations to O(1) by hoisting invariant

`

expressions to the pre-header, and PRE then identifies expressions redundant across all paths and eliminates them entirely, ultimately enabling the compiler to replace a 10-iteration loop with a single constant.

`

6.Induction-Variable Strength Reduction and Linear Function Test Replacement (LFTR)

1. The original program

```
void test(int *arr, int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = i * 8;
    }
}
```

2. The assembly code of the original program. (with – O0 option).

```
test:
        push    rbp
        mov     rbp, rsp
        mov     QWORD PTR [rbp-24], rdi
        mov     DWORD PTR [rbp-28], esi
        mov     DWORD PTR [rbp-4], 0
        jmp     .L2
.L3:
        mov     eax, DWORD PTR [rbp-4]
        cdqe
        lea     rdx, [0+rax*4]
        mov     rax, QWORD PTR [rbp-24]
        add     rax, rdx
        mov     edx, DWORD PTR [rbp-4]
        sal     edx, 3
        mov     DWORD PTR [rax], edx
        add     DWORD PTR [rbp-4], 1
.L2:
        mov     eax, DWORD PTR [rbp-4]
        cmp     eax, DWORD PTR [rbp-28]
        jl      .L3
        nop
        nop
        pop     rbp
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\ivsr_O0.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 1
Milliseconds      : 500
Ticks             : 15002880
TotalDays         : 1.73644444444444E-05
TotalHours        : 0.000416746666666667
TotalMinutes      : 0.0250048
TotalSeconds      : 1.500288
TotalMilliseconds : 1500.288
```

3. The assembly code of the optimized program (with – O1 option).

```
test:
        test    esi, esi
        jle     .L1
        movsx   rsi, esi
        mov     eax, 0
.L3:
        lea     edx, [0+rax*8]
        mov     DWORD PTR [rdi+rax*4], edx
        add     rax, 1
        cmp     rax, rsi
        jne     .L3
.L1:
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\ivsr_O1.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 1
Milliseconds      : 24
Ticks             : 10245334
TotalDays         : 1.1858025462963E-05
TotalHours        : 0.000284592611111111
TotalMinutes      : 0.0170755566666667
TotalSeconds      : 1.0245334
TotalMilliseconds : 1024.5334
```

`

4. The assembly code of the optimized program (with – O2 option).

```
test:
        test    esi, esi
        jle     .L1
        movsx   rsi, esi
        xor     eax, eax
.L3:
        lea     edx, [0+rax*8]
        mov     DWORD PTR [rdi+rax*4], edx
        add     rax, 1
        cmp     rsi, rax
        jne     .L3
.L1:
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\ivsr_O2.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 573
Ticks             : 5732470
TotalDays         : 6.63480324074074E-06
TotalHours        : 0.000159235277777778
TotalMinutes      : 0.00955411666666667
TotalSeconds      : 0.573247
TotalMilliseconds : 573.247
```

5. The assembly code of the optimized program (with – O3 option).

```
test:
        mov     ecx, esi
        test    esi, esi
        jle     .L1
        lea     eax, [rsi-1]
        cmp     eax, 2
        jbe     .L6
```

`

```
        mov     edx, esi
        movdqa  xmm1, XMMWORD PTR .LC0[rip]
        mov     esi, 4
        mov     rax, rdi
        shr     edx, 2
        movd    xmm2, esi
        sal     rdx, 4
        pshufd  xmm2, xmm2, 0
        add     rdx, rdi
.L4:
        movdqa  xmm0, xmm1
        add     rax, 16
        paddd   xmm1, xmm2
        pslld   xmm0, 3
        movups  XMMWORD PTR [rax-16], xmm0
        cmp     rax, rdx
        jne     .L4
        mov     eax, ecx
        and     eax, -4
        test    cl, 3
        je      .L9
.L3:
        movsx   rsi, eax
        lea     edx, [0+rax*8]
        mov     DWORD PTR [rdi+rsi*4], edx
        lea     r8, [0+rsi*4]
        lea     esi, [rax+1]
        cmp     ecx, esi
        jle     .L1
        lea     esi, [rdx+8]
        add     eax, 2
        mov     DWORD PTR [rdi+4+r8], esi
        cmp     eax, ecx
        jge     .L1
        add     edx, 16
        mov     DWORD PTR [rdi+8+r8], edx
.L1:
        ret
.L9:
        ret
.L6:
        xor     eax, eax
        jmp     .L3
.LC0:
```

`

```
        .long   0
        .long   1
        .long   2
        .long   3
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\ivsr_O3.exe }


 Days              : 0
 Hours             : 0
 Minutes           : 0
 Seconds           : 0
 Milliseconds      : 175
 Ticks             : 1756973
 TotalDays         : 2.03353356481481E-06
 TotalHours        : 4.88048055555556E-05
 TotalMinutes      : 0.00292828833333333
 TotalSeconds      : 0.1756973
 TotalMilliseconds : 175.6973
```

6. The assembly code of the optimized program (with – Os option).

```
test:
        xor     eax, eax
.L2:
        cmp     esi, eax
        jle     .L5
        lea     edx, [0+rax*8]
        mov     DWORD PTR [rdi+rax*4], edx
        inc     rax
        jmp     .L2
.L5:
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\ivsr_Os.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 972
Ticks             : 9729532
TotalDays         : 1.12610324074074E-05
TotalHours        : 0.000270264777777778
TotalMinutes      : 0.0162158866666667
TotalSeconds      : 0.9729532
TotalMilliseconds : 972.9532
```

7. Inference:

At the -O0 optimization level, the compiler performs a straightforward translation of the loop without analyzing relationships between variables inside the loop body. In every iteration, the value of the expression i * 8 is explicitly recomputed using a shift operation (sal edx, 3) based on the loop induction variable i. Since this computation is performed repeatedly for each iteration, it introduces unnecessary arithmetic overhead even though the value of i follows a predictable linear pattern.

With optimization enabled (-O1 and above), the compiler performs loop analysis to identify that i is a **basic induction variable**, meaning it increases by a constant value of 1 in each iteration of the loop. Consequently, the derived expression i * 8 is recognized as a **derived induction variable**, which increases by a constant value of 8 in every iteration.

Instead of recalculating the multiplication during each loop iteration, the compiler uses this linear relationship:

$$(i+1)\times 8=(i\times 8)+8$$

Using this observation, the multiplication operation inside the loop is replaced by an incremental addition of 8. This transformation reduces the strength of the operation from multiplication to addition,

`

which is computationally less expensive. As a result, the number of arithmetic operations executed at runtime is reduced, leading to improved instruction throughput and better utilization of processor resources.

Additionally, optimized versions maintain loop variables in registers rather than repeatedly accessing memory, which further reduces memory traffic and pipeline delays.

At the -O3 optimization level, the compiler combines strength reduction with additional loop optimizations such as loop unrolling and SIMD vectorization. Multiple values of the derived induction variable are computed simultaneously using packed vector instructions (e.g., paddd, pslld), enabling parallel execution of several iterations in a single instruction cycle.

Overall, induction variable strength reduction minimizes redundant arithmetic computations inside loops by converting multiplicative updates into simpler additive updates, thereby reducing instruction latency, loop overhead, and improving runtime performance, especially for loops with large iteration counts.

`

7.Inter-Procedural Function Inlining and Tail-Call Optimization (TCO)

1. The original program

```
int add(int a, int b) {
return a + b;
}

int wrapper(int x) {
return add(x, 10);
}

int tail(int n, int acc) {
if (n == 0) return acc;
return tail(n - 1, acc + n);
}
```

2. The assembly code of the optimized program (with – O0 option).

```
add:
push rbp
mov rbp, rsp
mov DWORD PTR [rbp-4], edi
mov DWORD PTR [rbp-8], esi
mov edx, DWORD PTR [rbp-4]
mov eax, DWORD PTR [rbp-8]
add eax, edx
pop rbp
ret
wrapper:
push rbp
mov rbp, rsp
sub rsp, 8
mov DWORD PTR [rbp-4], edi
mov eax, DWORD PTR [rbp-4]
mov esi, 10
mov edi, eax
call add
leave
ret
tail:
push rbp
mov rbp, rsp
```

`

```
sub rsp, 16
mov DWORD PTR [rbp-4], edi
mov DWORD PTR [rbp-8], esi
cmp DWORD PTR [rbp-4], 0
jne .L6
mov eax, DWORD PTR [rbp-8]
jmp .L7
.L6:
mov edx, DWORD PTR [rbp-8]
mov eax, DWORD PTR [rbp-4]
add edx, eax
mov eax, DWORD PTR [rbp-4]
sub eax, 1
mov esi, edx
mov edi, eax
call tail
.L7:
leave
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\inline_00.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 30
Ticks             : 305199
TotalDays         : 3.53239583333333E-07
TotalHours        : 8.47775E-06
TotalMinutes      : 0.000508665
TotalSeconds      : 0.0305199
TotalMilliseconds : 30.5199
```

3. The assembly code of the optimized program (with – O1 option).

```
add:
lea eax, [rdi+rsi]
ret
wrapper:
lea eax, [rdi+10]
ret
tail:
```

`

```
mov eax, esi
test edi, edi
jne .L9
ret
.L9:
sub rsp, 8
lea esi, [rdi+rsi]
sub edi, 1
call tail
add rsp, 8
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\inline_01.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 31
Ticks             : 316462
TotalDays         : 3.66275462962963E-07
TotalHours        : 8.79061111111111E-06
TotalMinutes      : 0.000527436666666667
TotalSeconds      : 0.0316462
TotalMilliseconds : 31.6462
```

4. The assembly code of the optimized program (with – O2 option).

```
add:
lea eax, [rdi+rsi]
ret
wrapper:
lea eax, [rdi+10]
ret
tail:
mov eax, esi
test edi, edi
je .L8
lea edx, [rdi-1]
test dil, 1
je .L5
add eax, edi
mov edi, edx
test edx, edx
```

`

```
je .L19
.L5:
lea eax, [rax-1+rdi*2]
sub edi, 2
jne .L5
.L8:
ret
.L19:
ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\inline_02.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 39
Ticks             : 392282
TotalDays         : 4.54030092592593E-07
TotalHours        : 1.08967222222222E-05
TotalMinutes      : 0.000653803333333333
TotalSeconds      : 0.0392282
TotalMilliseconds : 39.2282
```

5. The assembly code of the optimized program (with – O3 option).

```
add:
lea eax, [rdi+rsi]
ret
wrapper:
lea eax, [rdi+10]
ret
tail:
test edi, edi
je .L18
lea edx, [rdi-1]
cmp edx, 3
jbe .L11
movd xmm4, edi
mov eax, edi
mov ecx, -4
xor edx, edx
pshufd xmm0, xmm4, 0
```

`

```
movd xmm3, ecx
shr eax, 2
pxor xmm1, xmm1
paddd xmm0, XMMWORD PTR .LC0[rip]
pshufd xmm3, xmm3, 0
.L8:
movdqa xmm2, xmm0
add edx, 1
paddd xmm0, xmm3
paddd xmm1, xmm2
cmp edx, eax
jne .L8
movdqa xmm0, xmm1
psrldq xmm0, 8
paddd xmm1, xmm0
movdqa xmm0, xmm1
psrldq xmm0, 4
paddd xmm1, xmm0
movd edx, xmm1
lea eax, [rsi+rdx]
mov edx, edi
and edx, -4
test dil, 3
je .L4
sub edi, edx
lea edx, [rdi-1]
.L6:
add eax, edi
test edx, edx
je .L4
add eax, edx
mov edx, edi
sub edx, 2
je .L4
add eax, edx
lea eax, [rdi-3+rax]
.L4:
ret
.L18:
mov eax, esi
ret
.L11:
mov eax, esi
jmp .L6
```

`

```
.LC0:
.long 0
.long -1
.long -2
.long -3
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\inline_03.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 32
Ticks             : 320510
TotalDays         : 3.70960648148148E-07
TotalHours        : 8.90305555555556E-06
TotalMinutes      : 0.000534183333333333
TotalSeconds      : 0.032051
TotalMilliseconds : 32.051
```

6. The assembly code of the optimized program (with – Os option).

```
add:
lea eax, [rdi+rsi]
ret
wrapper:
lea eax, [rdi+10]
ret
tail:
mov eax, esi
.L5:
test edi, edi
je .L6
add eax, edi
dec edi
jmp .L5
.L6:
ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\inline_03.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 32
Ticks             : 320510
TotalDays         : 3.70960648148148E-07
TotalHours        : 8.90305555555556E-06
TotalMinutes      : 0.000534183333333333
TotalSeconds      : 0.032051
TotalMilliseconds : 32.051
```

7. Inference: Analysis of Inter-Procedural Function Inlining and Tail-Call Optimization (TCO)

At -O0, all function boundaries are preserved. wrapper invokes add via an explicit call, and tail performs true recursion with stack frame allocation on each invocation. This results in high overhead due to repeated prologue/epilogue execution, stack traffic, and control-flow disruption.

At -O1, inter-procedural inlining is applied: add is fully inlined into wrapper, reducing it to a single lea instruction. This eliminates call overhead and enables constant folding (x + 10). However, the recursive tail is only partially optimized—although some simplification occurs, it still uses call tail, meaning stack growth persists.

At -O2, more aggressive control-flow and loop optimizations are applied. The tail-recursive structure is transformed into a non-recursive form, eliminating repeated function calls. The compiler restructures the computation using arithmetic transformations (strength reduction and partial unrolling), significantly reducing branch frequency and improving instruction-level parallelism. Stack usage becomes constant, and execution becomes more cache-friendly.

At -O3, the optimizer further applies aggressive transformations such as SIMD vectorization and heavy loop restructuring. The tail function is converted into a vectorized reduction using SSE instructions (xmm registers), processing multiple iterations per cycle. While this

`

increases throughput, it introduces substantial code expansion, higher register pressure, and more complex control flow.

At -Os, the compiler prioritizes code size. The tail recursion is converted into a simple loop without vectorization or aggressive unrolling, achieving a balance between eliminating recursion (via TCO-like transformation) and minimizing instruction footprint.

Feature	-O2	-O3
Instruction Cache	Small, tight loops fit entirely in the L1 i-cache.	Large, unrolled vectorized code can cause "cache misses."
Pipeline Latency	Simple instructions have low "bypass" latency.	Vector instructions (SSE/AVX) often have higher latency/startup costs.
Binary Size	Minimal. Great for embedded or memory-constrained systems.	"Code Bloat." The binary grows because of loop unrolling.
The "Small N" Problem	Fast from the first iteration.	Slow for small n because the setup time for the xmm registers is longer than just doing the math.

`

8.Loop Unrolling, Peeling, and Interchange

1. Original Program (C):

```c
void test(int n, int m, int a[n][m]) {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            a[j][i] = i + j;
        }
    }
}
```

2. The assembly code of the optimized program (with – O0 option).

```asm
test:
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-20], edi
        mov     DWORD PTR [rbp-24], esi
        mov     QWORD PTR [rbp-32], rdx
        mov     eax, DWORD PTR [rbp-24]
        movsx   rdx, eax
        sub     rdx, 1
        mov     QWORD PTR [rbp-16], rdx
        mov     DWORD PTR [rbp-4], 0
        jmp     .L2
.L5:
        mov     DWORD PTR [rbp-8], 0
        jmp     .L3
.L4:
        mov     edx, DWORD PTR [rbp-8]
        movsx   rcx, edx
        movsx   rdx, eax
        imul    rdx, rcx
        lea     rcx, [0+rdx*4]
        mov     rdx, QWORD PTR [rbp-32]
        add     rcx, rdx
        mov     esi, DWORD PTR [rbp-4]
        mov     edx, DWORD PTR [rbp-8]
```

`

```
        add     esi, edx
        mov     edx, DWORD PTR [rbp-4]
        movsx   rdx, edx
        mov     DWORD PTR [rcx+rdx*4], esi
        add     DWORD PTR [rbp-8], 1
.L3:
        mov     edx, DWORD PTR [rbp-8]
        cmp     edx, DWORD PTR [rbp-24]
        jl      .L4
        add     DWORD PTR [rbp-4], 1
.L2:
        mov     edx, DWORD PTR [rbp-4]
        cmp     edx, DWORD PTR [rbp-20]
        jl      .L5
        nop
        nop
        pop     rbp
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\loop_00.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 42
Ticks             : 421732
TotalDays         : 4.88115740740741E-07
TotalHours        : 1.17147777777778E-05
TotalMinutes      : 0.000702886666666667
TotalSeconds      : 0.0421732
TotalMilliseconds : 42.1732
```

3. The assembly code of the optimized program (with – O1 option).

```
test:
        movsx   r9, esi
        test    edi, edi
        jle     .L1
        mov     r8d, esi
```

`

```
        sal     r9, 2
        mov     r10d, 0
        jmp     .L3
.L4:
        mov     DWORD PTR [rcx], eax
        add     eax, 1
        add     rcx, r9
        cmp     eax, r8d
        jne     .L4
.L6:
        add     r10d, 1
        add     rdx, 4
        add     r8d, 1
        cmp     edi, r10d
        je      .L1
.L3:
        mov     eax, r10d
        mov     rcx, rdx
        test    esi, esi
        jg      .L4
        jmp     .L6
.L1:
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\loop_01.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 33
Ticks             : 333543
TotalDays         : 3.86045138888889E-07
TotalHours        : 9.26508333333333E-06
TotalMinutes      : 0.000555905
TotalSeconds      : 0.0333543
TotalMilliseconds : 33.3543
```

4. The assembly code of the optimized program (with – O2 option).

```
test:
```

`

```
        movsx   r9, esi
        test    edi, edi
        jle     .L1
        mov     r8d, esi
        sal     r9, 2
        xor     r10d, r10d
.L3:
        test    esi, esi
        jg      .L5
        add     r10d, 1
        add     rdx, 4
        add     r8d, 1
        cmp     edi, r10d
        jne     .L3
.L1:
        ret
.L13:
        add     r10d, 1
        add     rdx, 4
        add     r8d, 1
        cmp     edi, r10d
        je      .L12
.L5:
        mov     eax, r10d
        mov     rcx, rdx
.L4:
        mov     DWORD PTR [rcx], eax
        add     eax, 1
        add     rcx, r9
        cmp     eax, r8d
        jne     .L4
        jmp     .L13
.L12:
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\loop_02.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 54
Ticks             : 549067
TotalDays         : 6.35494212962963E-07
TotalHours        : 1.52518611111111E-05
TotalMinutes      : 0.000915111666666667
TotalSeconds      : 0.0549067
TotalMilliseconds : 54.9067
```

5. The assembly code of the optimized program (with – O3 option).

```
test:
        movsx   r8, esi
        test    edi, edi
        jle     .L1
        test    esi, esi
        jle     .L1
        sal     r8, 2
        xor     r9d, r9d
.L3:
        mov     eax, r9d
        mov     rcx, rdx
.L4:
        mov     DWORD PTR [rcx], eax
        add     eax, 1
        add     rcx, r8
        cmp     eax, esi
        jne     .L4
        add     r9d, 1
        add     rdx, 4
        add     esi, 1
        cmp     edi, r9d
        jne     .L3
.L1:
```

`

```
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\loop_03.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 28
Ticks             : 289376
TotalDays         : 3.34925925925926E-07
TotalHours        : 8.03822222222222E-06
TotalMinutes      : 0.000482293333333333
TotalSeconds      : 0.0289376
TotalMilliseconds : 28.9376
```

6. The assembly code of the optimized program (with – Os option).

```
test:
        movsx   r8, esi
        xor     eax, eax
        sal     r8, 2
.L2:
        cmp     eax, edi
        jge     .L7
        mov     r9, rdx
        xor     ecx, ecx
.L4:
        cmp     ecx, esi
        jge     .L8
        lea     r10d, [rcx+rax]
        inc     ecx
        mov     DWORD PTR [r9], r10d
        add     r9, r8
        jmp     .L4
.L8:
        inc     eax
        add     rdx, 4
        jmp     .L2
.L7:
```

`

```
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\loop_0s.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 37
Ticks             : 375356
TotalDays         : 4.34439814814815E-07
TotalHours        : 1.04265555555556E-05
TotalMinutes      : 0.000625593333333333
TotalSeconds      : 0.0375356
TotalMilliseconds : 37.5356
```

7. Inference:

At the -O0 optimization level, the nested loops execute strictly in the order specified by the programmer, resulting in inefficient memory access patterns. Since the program updates the array using a[j][i], elements are accessed column-wise in memory, whereas multi-dimensional arrays in C are stored in row-major order. This leads to non-contiguous memory accesses and frequent cache misses, thereby reducing spatial locality and overall performance.

With optimization enabled (-O1 and above), the compiler analyzes the loop structure and applies **loop interchange** by swapping the order of the nested loops. This ensures that array elements are accessed in contiguous memory locations, improving cache utilization and reducing memory latency.

Further, the compiler applies **loop peeling** by separating the initial few iterations of the loop that may not satisfy alignment or boundary conditions required for advanced optimizations. By executing these iterations independently before entering the main loop body, the compiler ensures that the remaining iterations can be optimized safely without additional conditional checks.

`

The remaining iterations are then subjected to **loop unrolling**, where multiple loop operations are executed within a single iteration. This reduces the number of branch instructions and loop control updates, thereby minimizing control overhead.

At higher optimization levels such as -O3, these transformations enable SIMD-based vectorization, allowing multiple elements to be processed simultaneously using vector registers. This improves instruction-level parallelism and significantly enhances execution efficiency by reducing both loop overhead and cache miss penalties.

`

9.SIMD Autovectorization and Superword-Level Parallelism (SLP)

1. Original Program (C):

```c
void test(int *a, int *b, int *c, int n) {
    for(int i = 0; i < n; i++) {
        a[i] = b[i] + c[i]; }
}
```

2. The assembly code of the optimized program (with – O0 option).

```asm
test:
        push    rbp
        mov     rbp, rsp
        mov     QWORD PTR [rbp-24], rdi
        mov     QWORD PTR [rbp-32], rsi
        mov     QWORD PTR [rbp-40], rdx
        mov     DWORD PTR [rbp-44], ecx
        mov     DWORD PTR [rbp-4], 0
        jmp     .L2
.L3:
        mov     eax, DWORD PTR [rbp-4]
        cdqe
        lea     rdx, [0+rax*4]
        mov     rax, QWORD PTR [rbp-32]
        add     rax, rdx
        mov     ecx, DWORD PTR [rax]
        mov     eax, DWORD PTR [rbp-4]
        cdqe
        lea     rdx, [0+rax*4]
        mov     rax, QWORD PTR [rbp-40]
        add     rax, rdx
        mov     edx, DWORD PTR [rax]
        mov     eax, DWORD PTR [rbp-4]
        cdqe
        lea     rsi, [0+rax*4]
        mov     rax, QWORD PTR [rbp-24]
        add     rax, rsi
        add     edx, ecx
```

`

```
        mov     DWORD PTR [rax], edx
        add     DWORD PTR [rbp-4], 1
.L2:
        mov     eax, DWORD PTR [rbp-4]
        cmp     eax, DWORD PTR [rbp-44]
        jl      .L3
        nop
        nop
        pop     rbp
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\simd_00.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 33
Ticks             : 337002
TotalDays         : 3.90048611111111E-07
TotalHours        : 9.36116666666667E-06
TotalMinutes      : 0.00056167
TotalSeconds      : 0.0337002
TotalMilliseconds : 33.7002
```

3. The assembly code of the optimized program (with – O1 option).

```
test:
        test    ecx, ecx
        jle     .L1
        movsx   rcx, ecx
        lea     r8, [0+rcx*4]
        mov     eax, 0
.L3:
        mov     ecx, DWORD PTR [rdx+rax]
        add     ecx, DWORD PTR [rsi+rax]
        mov     DWORD PTR [rdi+rax], ecx
        add     rax, 4
        cmp     rax, r8
        jne     .L3
.L1:
```

`

```
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\simd_01.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 39
Ticks             : 392487
TotalDays         : 4.54267361111111E-07
TotalHours        : 1.09024166666667E-05
TotalMinutes      : 0.000654145
TotalSeconds      : 0.0392487
TotalMilliseconds : 39.2487
```

4. The assembly code of the optimized program (with – O2 option).

```
test:
        test    ecx, ecx
        jle     .L1
        movsx   rcx, ecx
        xor     eax, eax
        lea     r8, [0+rcx*4]
.L3:
        mov     ecx, DWORD PTR [rdx+rax]
        add     ecx, DWORD PTR [rsi+rax]
        mov     DWORD PTR [rdi+rax], ecx
        add     rax, 4
        cmp     r8, rax
        jne     .L3
.L1:
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\simd_02.exe }
Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 35
Ticks             : 358248
TotalDays         : 4.14638888888889E-07
TotalHours        : 9.95133333333333E-06
TotalMinutes      : 0.00059708
TotalSeconds      : 0.0358248
TotalMilliseconds : 35.8248
```

5. The assembly code of the optimized program (with – O3 option).

```
test:
        test    ecx, ecx
        jle     .L1
        cmp     ecx, 1
        je      .L3
        lea     r8, [rsi+4]
        mov     rax, rdi
        sub     rax, r8
        cmp     rax, 8
        jbe     .L3
        lea     r8, [rdx+4]
        mov     rax, rdi
        sub     rax, r8
        cmp     rax, 8
        jbe     .L3
        lea     eax, [rcx-1]
        mov     r9d, ecx
        cmp     eax, 2
        jbe     .L11
        mov     r8d, ecx
        xor     eax, eax
        shr     r8d, 2
        sal     r8, 4
.L5:
        movdqu  xmm0, XMMWORD PTR [rsi+rax]
        movdqu  xmm2, XMMWORD PTR [rdx+rax]
        paddd   xmm0, xmm2
```

`

```
        movups  XMMWORD PTR [rdi+rax], xmm0
        add     rax, 16
        cmp     r8, rax
        jne     .L5
        mov     eax, ecx
        and     eax, -4
        mov     r8d, eax
        cmp     ecx, eax
        je      .L1
        sub     ecx, eax
        mov     r9d, ecx
        cmp     ecx, 1
        je      .L7
.L4:
        mov     ecx, r8d
        movq    xmm0, QWORD PTR [rsi+rcx*4]
        movq    xmm1, QWORD PTR [rdx+rcx*4]
        paddd   xmm0, xmm1
        movq    QWORD PTR [rdi+rcx*4], xmm0
        test    r9b, 1
        je      .L1
        and     r9d, -2
        add     eax, r9d
.L7:
        cdqe
        mov     edx, DWORD PTR [rdx+rax*4]
        add     edx, DWORD PTR [rsi+rax*4]
        mov     DWORD PTR [rdi+rax*4], edx
        ret
.L3:
        movsx   rcx, ecx
        xor     eax, eax
        sal     rcx, 2
.L9:
        mov     r8d, DWORD PTR [rdx+rax]
        add     r8d, DWORD PTR [rsi+rax]
        mov     DWORD PTR [rdi+rax], r8d
        add     rax, 4
        cmp     rax, rcx
        jne     .L9
.L1:
        ret
.L11:
        xor     r8d, r8d
```

`

```
        xor     eax, eax
        jmp     .L4
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\simd_03.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 51
Ticks             : 511246
TotalDays         : 5.91719907407407E-07
TotalHours        : 1.42012777777778E-05
TotalMinutes      : 0.000852076666666667
TotalSeconds      : 0.0511246
TotalMilliseconds : 51.1246
```

6. The assembly code of the optimized program (with – Os option).

```
test:
        xor     eax, eax
.L2:
        cmp     ecx, eax
        jle     .L5
        mov     r8d, DWORD PTR [rdx+rax*4]
        add     r8d, DWORD PTR [rsi+rax*4]
        mov     DWORD PTR [rdi+rax*4], r8d
        inc     rax
        jmp     .L2
.L5:
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\simd_0s.exe }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 30
Ticks             : 307182
TotalDays         : 3.55534722222222E-07
TotalHours        : 8.53283333333333E-06
TotalMinutes      : 0.00051197
TotalSeconds      : 0.0307182
TotalMilliseconds : 30.7182
```

7. Inference:

At -O0, the loop executes in a purely scalar manner, where each iteration performs the addition of a single pair of array elements from b[i] and c[i] and stores the result in a[i]. This results in sequential execution with one arithmetic operation per iteration, limiting performance due to high loop overhead and underutilization of the processor’s data-parallel capabilities.

With optimization enabled at -O2 and above, the compiler detects that the operations performed in different loop iterations are independent and have no data dependencies. This allows the application of **SIMD autovectorization**, where multiple data elements are grouped together and processed simultaneously using vector instructions such as paddd.

This optimization utilizes **Superword-Level Parallelism (SLP)**, in which several scalar operations of the same type are combined into a single vector instruction operating on packed data elements. For instance, four integer additions can be performed in parallel within a single instruction cycle using SIMD registers.

Additionally, the compiler may apply loop peeling to handle leftover iterations that do not fit into vector-sized chunks. As a result, multiple elements are processed per iteration of the loop, reducing the total number of loop iterations and minimizing branch overhead.

`

At -Os, the compiler avoids vectorization in order to maintain smaller code size, even though it may result in reduced execution speed.

Overall, SIMD autovectorization combined with SLP improves runtime performance by exploiting data-level parallelism, reducing instruction count, and enhancing processor throughput.

10.Inter-Procedural Alias and Side-Effect Analysis

1. Original Program (C):

```c
void update(int *x, int *y) {
    *x = *x + 5;
    *y = *y + 10;
}
void test(int *a, int *b) {
update(a, b);
}
```

2. The assembly code of the optimized program (with – O0 option).

```asm
update:
        push    rbp
        mov     rbp, rsp
        mov     QWORD PTR [rbp-8], rdi
        mov     QWORD PTR [rbp-16], rsi
        mov     rax, QWORD PTR [rbp-8]
        mov     eax, DWORD PTR [rax]
        lea     edx, [rax+5]
        mov     rax, QWORD PTR [rbp-8]
        mov     DWORD PTR [rax], edx
        mov     rax, QWORD PTR [rbp-16]
        mov     eax, DWORD PTR [rax]
        lea     edx, [rax+10]
        mov     rax, QWORD PTR [rbp-16]
        mov     DWORD PTR [rax], edx
        nop
        pop     rbp
        ret
test:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
```

`

```
        mov     QWORD PTR [rbp-8], rdi
        mov     QWORD PTR [rbp-16], rsi
        mov     rdx, QWORD PTR [rbp-16]
        mov     rax, QWORD PTR [rbp-8]
        mov     rsi, rdx
        mov     rdi, rax
        call    update
        nop
        leave
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\alias_00.exe }
Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 35
Ticks             : 352867
TotalDays         : 4.0841087962963E-07
TotalHours        : 9.80186111111111E-06
TotalMinutes      : 0.000588111666666667
TotalSeconds      : 0.0352867
TotalMilliseconds : 35.2867
```

3. The assembly code of the optimized program (with – O1 option).

```
update:
        add     DWORD PTR [rdi], 5
        add     DWORD PTR [rsi], 10
        ret
test:
        call    update
        ret
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\alias_01.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 45
Ticks             : 458411
TotalDays         : 5.30568287037037E-07
TotalHours        : 1.27336388888889E-05
TotalMinutes      : 0.000764018333333333
TotalSeconds      : 0.0458411
TotalMilliseconds : 45.8411
```

4. The assembly code of the optimized program (with – O2 option).

```
update:
        add     DWORD PTR [rdi], 5
        add     DWORD PTR [rsi], 10
        ret
test:
        add     DWORD PTR [rdi], 5
        add     DWORD PTR [rsi], 10
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\alias_02.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 36
Ticks             : 368329
TotalDays         : 4.26306712962963E-07
TotalHours        : 1.02313611111111E-05
TotalMinutes      : 0.000613881666666667
TotalSeconds      : 0.0368329
TotalMilliseconds : 36.8329
```

5. The assembly code of the optimized program (with – O3 option).

`

```
update:
        add     DWORD PTR [rdi], 5
        add     DWORD PTR [rsi], 10
        ret
test:
        add     DWORD PTR [rdi], 5
        add     DWORD PTR [rsi], 10
        ret
```

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\alias_03.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 32
Ticks             : 323485
TotalDays         : 3.74403935185185E-07
TotalHours        : 8.98569444444444E-06
TotalMinutes      : 0.000539141666666667
TotalSeconds      : 0.0323485
TotalMilliseconds : 32.3485
```

6. The assembly code of the optimized program (with – Os option).

```
update:
        add     DWORD PTR [rdi], 5
        add     DWORD PTR [rsi], 10
        ret
test:
        jmp     update
```

`

Execution Time:

```
PS C:\Users\TARUN\Desktop\Code Optimization> Measure-Command { .\exe\alias_0s.exe }


Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 28
Ticks             : 287851
TotalDays         : 3.3316087962963E-07
TotalHours        : 7.99586111111111E-06
TotalMinutes      : 0.000479751666666667
TotalSeconds      : 0.0287851
TotalMilliseconds : 28.7851
```

7. Inference:

At the -O0 optimization level, the compiler handles function calls conservatively because pointer parameters passed to a function may potentially refer to the same memory location. In the given example, both parameters x and y are pointers, and the compiler cannot initially assume that they refer to different memory addresses. If they were aliased (i.e., pointing to the same memory location), modifying one could affect the value accessed through the other. Therefore, the compiler preserves the function call structure and performs memory loads and stores separately to maintain program correctness.

With optimization enabled (-O1 and above), the compiler performs **inter-procedural analysis**, where it examines the behavior of the called function (update) across function boundaries. It checks for:

- possible pointer aliasing between x and y
- memory locations modified by the function
- absence of global side effects

After analyzing the function body, the compiler determines that the updates to *x and *y are independent operations and do not rely on shared state or external memory effects. Since the function has no

`

observable side effects beyond modifying its input pointers, it becomes safe to inline the function into the caller.

At higher optimization levels such as -O2 and -O3, this allows the compiler to eliminate the function call entirely and directly apply the memory updates within the test function. This removes the overhead of:

- stack frame creation
- parameter passing
- return address handling
- control transfer during call–return

As a result, both memory updates are performed directly using register-based addressing inside the caller, improving execution efficiency by reducing instruction count, memory access latency, and function call overhead.

In some cases, when full inlining is not performed but aliasing is proven safe, the compiler may also replace the call–return sequence with a direct jump (jmp update), enabling tail-call optimization and avoiding unnecessary stack usage.

Overall, inter-procedural alias and side-effect analysis enables the compiler to safely optimize across function boundaries by ensuring that pointer-based memory updates do not interfere with each other, thereby improving runtime performance without altering program semantics.

`

Conclusion

Through the systematic analysis of ten distinct compiler optimization techniques using the GCC compiler, this case study highlights the profound impact of the Optimizer (Middle-End) on software performance. At the baseline -O0 level, the compiler strictly prioritizes rapid compilation and predictable debugging, resulting in literal, unoptimized translations characterized by redundant memory access, heavy register pressure, and bloated stack frames.

However, as optimization levels scale (-O1 through -O3), the compiler acts as an advanced code restructuring engine. Techniques like Global Dead-Code Elimination (DCE), Loop-Invariant Code Motion (LICM), and Constant Propagation strip away redundant computations and hoist invariants, drastically reducing the dynamic instruction count. Memory bottlenecks are alleviated through Loop Interchange and Alias Analysis, which maximize cache utilization and spatial locality. Furthermore, high-level optimizations like Function Inlining and SIMD Autovectorization transform control flow and leverage modern hardware capabilities, replacing sequential operations with parallel execution. Finally, the -Os flag effectively demonstrates the compiler's ability to balance these architectural improvements while aggressively minimizing the binary footprint. Ultimately, this analysis confirms that modern compiler optimization is not merely about translating code, but fundamentally rewriting it to achieve hardware synergy without altering programmer intent.